

Apprentice Project : Data Transfer Server

1 - Introduction [🔗](#)

The goal of this project is to develop a small web application to be used to simulate data being transferred over HTTP on a network.

(The application is to be used to simulate data transfer, in order to validate network metrics generated by OV.)

The project is full stack, with a UI part, a backend part, CI/CD, and infrastructure/ops part.

2 - Project Specification:

[🔗](#) The goal of this project is to develop a small web application to be used to simulate data being transferred over HTTP on a network.

The first menu, is the download menu, to allow simulating downloading data :

The screenshot displays the 'Data transfer server' web application. On the left is a blue sidebar with a folder icon and three menu items: 'Download' (highlighted with a blue underline), 'Upload', and 'Scenarios'. The main content area has a white background and is titled 'Fixed Size Download' at the top. Below this title, there are three rows of file information, each with a folder icon, a filename, a size, and a green 'Download' button. The files are 'File A' (1MB), 'File B' (10 MB), and 'File C' (100 MB). Below the 'Fixed Size Download' section is a 'Custom Size' section, which includes a text input field containing '100MB', a small control with '+' and '-' buttons, and a green 'Download' button. At the bottom of the main content area is a 'Continuous Download' section with a single green button labeled 'Go !'.

Fixed Size Download			
	File A	1MB	<button>Download</button>
	File B	10 MB	<button>Download</button>
	File C	100 MB	<button>Download</button>

Custom Size	
100MB	+ -
<button>Download</button>	

Continuous Download	
<button>Go !</button>	

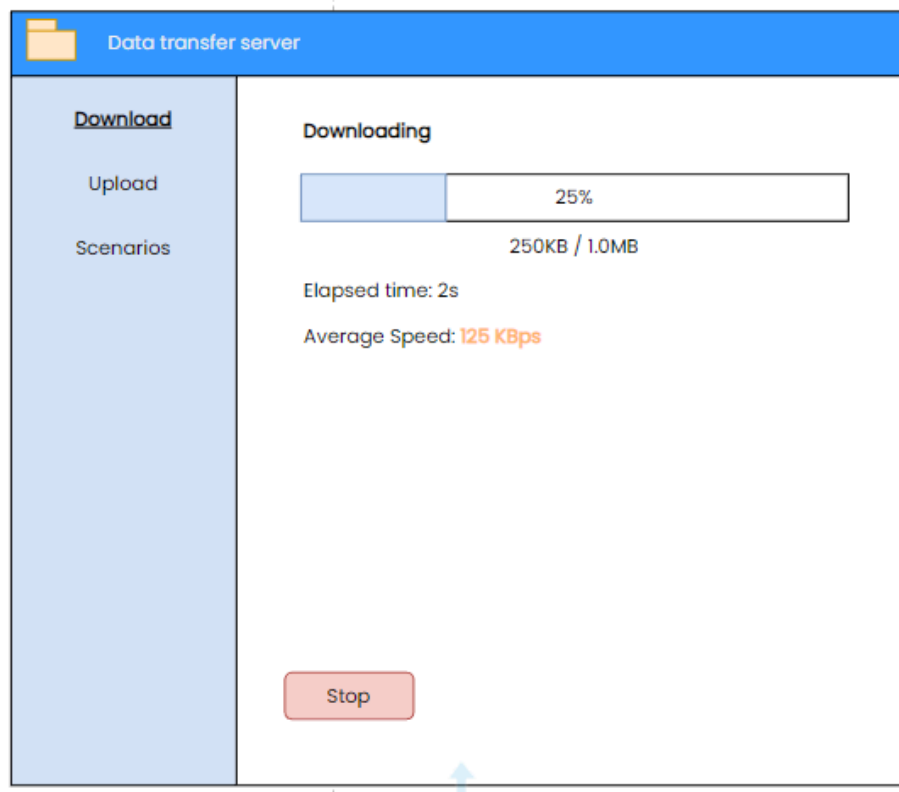
User, can use 3 buttons, to either trigger 1, 10, or 100MB download.

Or he can use a button to download a custom sized file. (In that case, the content of the file is random)

The data to be downloaded should be randomly generated bytes from the backend service.

Or it can be a continuous download, that will only stop on user input.

In continuous download mode, it is not expected to actually save the file on the end user computer, but just to trigger data transfer between the server and the client browser is enough.



When a download is launched, user can follow the download with a progress bar. Many values are expected to be displayed:

- A progress percentage
- The data retrieved until now
- The total data to be retrieved (not in continuous mode)
- The elapsed time
- The average speed

With a stop button, the user can interrupt the download.

Upload menu:

Exactly the same principle as the download menu, but the other way.

Download

Upload

Scenarios

Data transfer server

Custom File

Drag your file here.



Parcourir

Fixed Size Upload



1MB

Upload



10 MB

Upload



100 MB

Upload

Custom Size

100MB

+

-

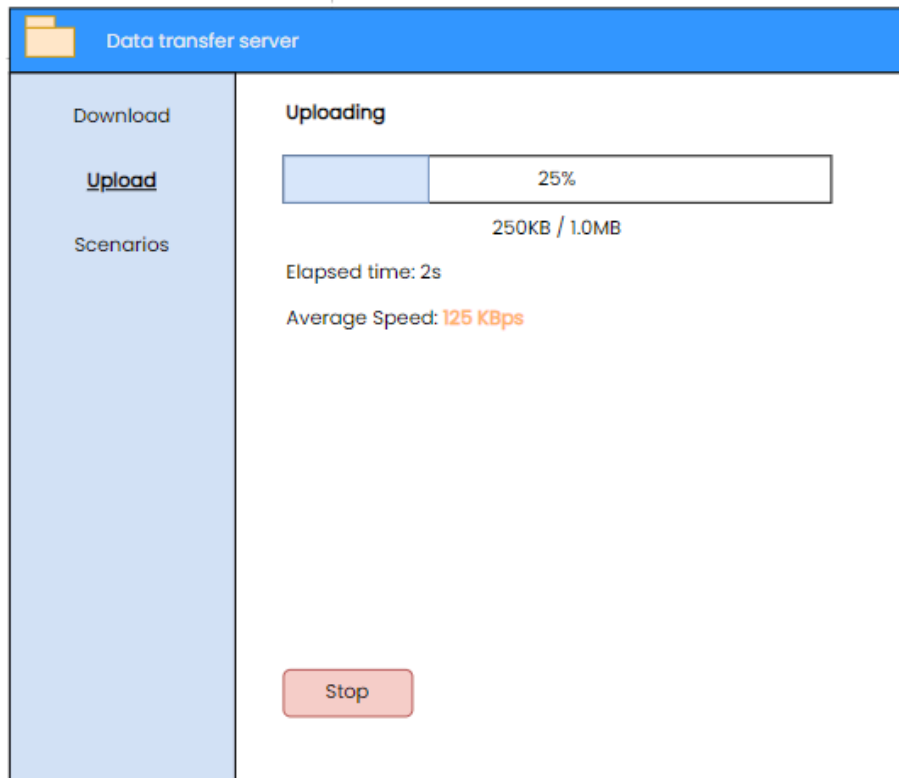
Upload

Continuous Upload

Go !

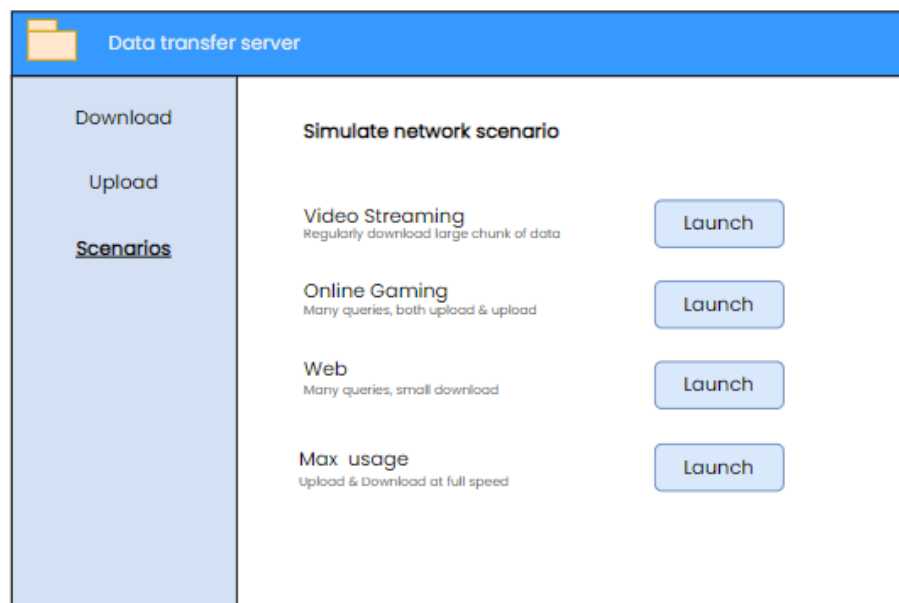
In case user click on 1MB, 10MB, 100MB, or Custom size upload, it will upload randomly generated bytes (from the browser) to the server. Not an actual file.

There is also drag'n'drop zone, to easily send a specific file:

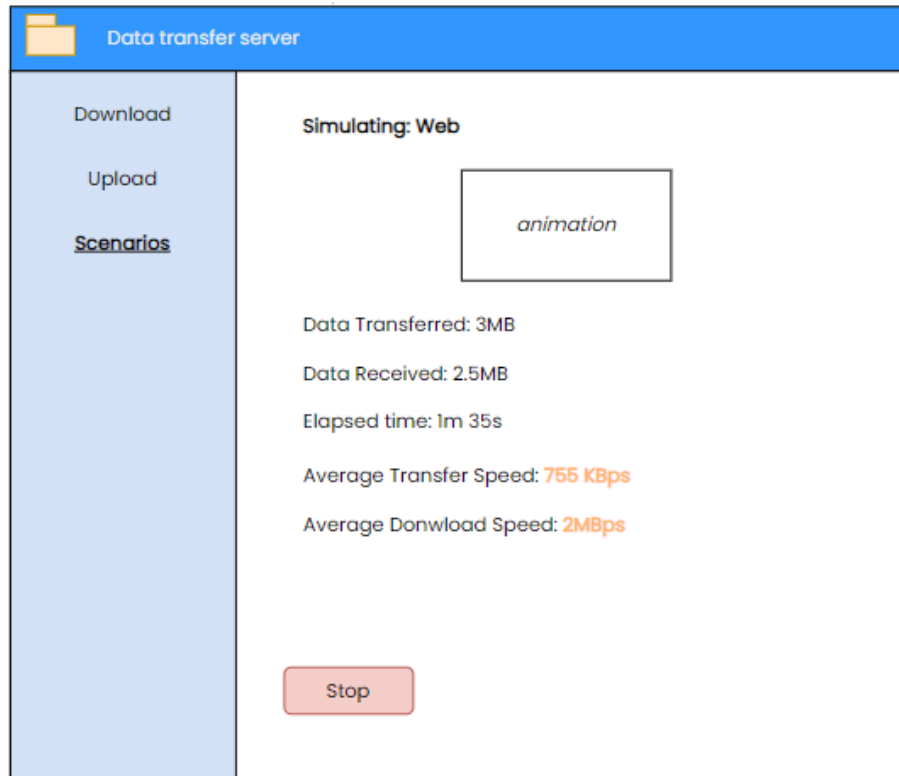


Simulation menu:

From this menu, the user can launch multiple scenarios:



Once the scenario is launched, it will open a dedicated simulation view.



Scenarios should be:

Video Streaming:

- Regularly download large chunk of data:
 - Query 10MB chunk every 10seconds
- Online Gaming
 - Query 1kb every second
 - Send 1kb every second
- Web:
 - Every 30 seconds
 - Query 10 times 50KB in parrallel (Simulate opening a website, and all calls to retrieve html,js,css)
 - Upload 10KB
- Max Usage:
 - Continuously Download chunk of data (like “continuous download”)
 - Continuously Upload chunk of data (like “continuous upload”)

Others requirements: [🔗](#)

⚠️ The project user interface should be mobile friendly.

3 - Your tasks:



- Setup a git repository on our Gitlab (<https://scm.netaos.us/>)
- Write the backend code (In Go or Java)
- Write the frontend code (In Typescript / VueJS 3)
- Package the solution as a Docker image as small as possible
- Provide a configuration file to deploy it on Kubernetes
- Implement CI/CD on your Gitlab repository, on each commit pushed to the repository:
 - Run tests
 - Build the Docker Image
 - Run a vulnerability scan of the Docker image using Trivy
 - Deploy the image on a development Kubernetes cluster